

VidFetch: seguimiento de robots en video

Uso, funcionamiento y código de la aplicación

Dylan Frigerio
ITBA

October 8, 2026

Abstract

VidFetch mide la posición, la velocidad, la orientación y la velocidad angular de N robots circulares filmados desde arriba. Detecta cada robot con un filtro adaptado a su forma (disco claro rodeado de un anillo oscuro) y `trackpy.locate`, lo sigue con N fijo mediante asignación óptima y predicción, deriva la velocidad con un filtro de Savitzky–Golay y obtiene el giro comparando firmas polares en el dominio de Fourier. Este informe explica para qué sirve, cómo está organizado, qué bibliotecas usa y qué hace cada parte del código. Los resultados de los ensayos están en un informe aparte.

Índice

1	Qué es y para qué sirve	3
1.1	Requisitos y ejecución	3
1.2	Organización del código	3
1.3	La interfaz	4
1.4	Archivos que lee y escribe	5
2	Bibliotecas utilizadas	5
3	Escala de tiempo: videos acelerados	7
4	Metodología de medición	8
4.1	Flujo de procesamiento	9
4.2	Sistema de referencia y unidades	9
4.3	El recinto: detección, marcado manual y escala	10
4.4	Calibración automática del tamaño	12
4.5	Detección	13
4.6	Seguimiento con N fijo	15
4.7	Velocidad	16
4.8	Orientación y velocidad angular	17
4.9	Recorte temporal	20
4.10	Incertidumbres y limitaciones	20
5	Procedimiento y exportaciones	21
5.1	Instalación	21
5.2	Analizar un video con la interfaz	21
5.3	Definir el recinto a mano	22
5.4	Exportaciones	22
5.5	Reexportar sin la interfaz: <code>reexportar.py</code>	24
5.6	Verificar el giro: <code>verificar_giro.py</code>	24
5.7	Generar el informe de ensayos	25
5.8	Rehacer todo desde cero	25

1 Qué es y para qué sirve

VidFetch es una aplicación de escritorio en Python que mide, a partir de un video filmado desde arriba, el movimiento de N robots circulares dentro de una arena. Para cada robot y cada cuadro entrega la posición del centro, el vector velocidad, la orientación acumulada θ y la velocidad angular ω , en unidades SI y en tiempo real aunque el video esté acelerado. Las posiciones se dan con origen en el centro del recinto circular (detectado automáticamente o marcado a mano), que además fija la escala espacial.

Usos previstos:

- **Preparar el video:** recortar el tramo útil y el rectángulo de la arena, y guardar el recorte (archivos `VC_`).
- **Probar la detección** sobre cuadros sueltos y ajustar parámetros a mano o con la calibración automática.
- **Seguir N robots** fijos durante todo el ensayo, revisar los cuadros dudosos y corregir centros a mano.
- **Ubicar el recinto** en cada cuadro (centro y radio), para medir respecto de su centro y convertir a milímetros con su diámetro conocido.
- **Exportar** los datos (CSV de todos los robots, de un robot o un archivo por robot), un resumen por robot, el video anotado y la sesión de análisis para retomarla sin volver a procesar.
- **Reexportar sin interfaz** una sesión guardada (`tools/reexportar.py`).
- **Generar los informes** de ensayos a partir de los CSV (sección 5).

1.1 Requisitos y ejecución

Requiere Python 3.10 o superior y las bibliotecas de `requirements.txt` (sección 2). Desde la carpeta `software/vidfetch` del repositorio:

```
pip install -r requirements.txt
python main.py
```

Las tareas largas (análisis, exportaciones) corren en un hilo aparte, con barra de progreso y botón de cancelar, de modo que la ventana no se congela.

1.2 Organización del código

El código separa la lógica de cálculo (`core/`, sin dependencias de la interfaz) de la interfaz gráfica (`gui/`). Así, todo el cálculo puede usarse desde un script, y la interfaz solo llama a funciones de `core/` (tabla 1).

Tabla 1: Archivos del programa y su función (rutas relativas a `software/vidfetch`).

Archivo	Líneas	Función
<code>main.py</code>	18	punto de entrada: crea la aplicación Qt y la ventana principal
<code>core/models.py</code>	47	Roi (rectángulo) y TrimRange (tramo de cuadros)
<code>core/video_io.py</code>	169	lectura de video con acceso aleatorio, exportación de video recortado o anotado
<code>core/processing.py</code>	94	operaciones de imagen encadenables (grises, brillo/contraste, desenfoque, umbral, Canny)
<code>core/detection.py</code>	182	detección de robots: filtro de anillo, <code>trackpy.locate</code> , cobertura del anillo
<code>core/calibration.py</code>	230	estimación automática del radio del robot y de todos los umbrales
<code>core/analysis.py</code>	125	recorrido del video cuadro a cuadro (candidatos y recinto)
<code>core/tracking.py</code>	756	seguimiento con N fijo, anclas manuales, huecos, sesión <code>.npz</code> , dibujo
<code>core/rotation.py</code>	188	firmas polares y orientación acumulada θ (enlaces cortos y largos)
<code>core/arena.py</code>	717	recinto: detección, ajuste de círculos, seguimiento temporal, marcado manual, dibujo
<code>core/kinematics.py</code>	426	velocidad, ω , sistema del recinto, escala y tablas de exportación
<code>gui/main_window.py</code>	651	ventana, menús, diálogos de archivo, exportaciones y tareas en segundo plano
<code>gui/state.py</code>	71	estado compartido: video, tramo, rectángulo, escala de tiempo
<code>gui/tabs.py</code>	501	pestañas <i>Original</i> , <i>Recortar</i> y <i>Procesar</i>
<code>gui/tracking_tab.py</code>	729	pestaña <i>Seguimiento</i> (incluye el recinto y las exportaciones por robot)
<code>gui/player.py, frame_view.py</code>	362	reproductor y visor con zoom y edición del rectángulo
<code>gui/export_worker.py</code>	35	hilo de trabajo (<code>QThread</code>) para tareas largas
<code>tools/reexportar.py</code>	126	reexportar una sesión sin interfaz (recinto, CSV, resumen, por robot)
<code>tools/verificar_giro.py</code>	137	verificación visual de θ sobre el video
<i>En <code>software/analisis_video/</code>:</i>		
<code>generar_informe.py</code>	637	sección de resultados de un ensayo a partir de su CSV
<code>comparar_ensayos.py</code>	584	comparación entre ensayos y contraste con el sensor de presión

1.3 La interfaz

La ventana tiene cuatro pestañas que comparten el mismo video, tramo y rectángulo, guardados en un único objeto de estado (`gui/state.py`): lo que se define en una se refleja en las demás.

1. **Original:** reproduce el video y fija la escala de tiempo (sección 3). La etiqueta del reproductor muestra el cuadro y el tiempo real.
2. **Recortar:** tramo temporal (cuadro de inicio y fin) y rectángulo de análisis, que se dibuja o se mueve sobre la imagen; permite forzar un cuadrado de lado fijo y guardar el recorte.
3. **Procesar:** aplica operaciones de imagen y prueba la detección cuadro a cuadro (parámetros, botón *Calibrar tamaño automáticamente*). Exporta posiciones sueltas o vinculadas con `trackpy.link` y el video procesado.
4. **Seguimiento:** análisis completo con N fijo, lista de cuadros con problemas, corrección manual del centro, recinto (detección automática, marcado manual, diámetros), parámetros de

cinemática (ventana de suavizado, diámetro real, video espejado, fuente de la escala), superposición de velocidad, orientación y recinto, y exportaciones.

1.4 Archivos que lee y escribe

Tabla 2: Formatos de archivo.

Extensión	Sentido	Contenido
.mp4 .avi .mov .mkv .m4v .wmv	entrada	video a analizar (cualquier formato que lea OpenCV/FFmpeg)
.mp4 (mp4v), .avi (MJPG)	salida	video recortado, procesado o con el seguimiento dibujado
.csv	salida	posiciones de la pestaña <i>Procesar</i> ; seguimiento completo en formato largo o ancho; un robot; un archivo por robot; resumen por robot (tablas 7 and 8)
_info.json	salida	junto a cada CSV: recinto, escala, convenciones, κ , espejado, f_r
.npz	entrada y salida	sesión de análisis: candidatos, firmas de rotación, anclas manuales, recinto y parámetros (NumPy comprimido, sin <code>pickle</code>)
.tex, .csv, .png	salida	secciones de informe generadas por los scripts de <code>software/analisis_video/</code>

La sesión `.npz` se guarda con `numpy.savez_compressed` y se carga con `allow_pickle=False`: contiene solo arreglos numéricos y un texto JSON con los parámetros, por lo que abrir un archivo ajeno no ejecuta código.

2 Bibliotecas utilizadas

Tabla 3 resume qué se usa de cada biblioteca. La regla general es usar funciones compiladas (OpenCV, SciPy, NumPy vectorizado) para todo lo que se repite por píxel o por cuadro, y dejar en Python solo la lógica de decisión.

Tabla 3: Bibliotecas, funciones usadas y para qué. Versiones probadas entre paréntesis.

Biblioteca	Funciones	Uso
OpenCV (cv2 5.0)	VideoCapture, CAP_PROP_*	abrir el video, leer metadatos (cuadros, fps, tamaño) y saltar a un cuadro
	VideoWriter, VideoWriter_fourcc	escribir videos recortados o anotados
	cvtColor, resize (INTER_AREA)	pasar a grises y reducir la imagen a la escala de análisis
	filter2D	convolución con los núcleos de disco y anillo (filtro de anillo, sección 4.5)
	warpPolar	imagen en coordenadas polares alrededor de cada centro: cobertura del anillo y firma de rotación
	COLOR_BGR2HSV, threshold (Otsu), morphologyEx	máscara del anillo del recinto por tono y saturación (sección 4.3)
	HoughCircles, remap	centro provisorio del recinto; perfiles de saturación a lo largo de los rayos
	grab, retrieve	recorrer el video decodificando solo los cuadros donde se mide el recinto
trackpy (0.7)	GaussianBlur, threshold, Canny, convertScaleAbs, dilate	operaciones de la pestaña <i>Procesar</i> y calibración
	circle, line, arrowedLine, putText, ellipse	dibujo de la superposición y del video anotado
trackpy (0.7)	locate	máximos locales del mapa de respuesta con centroide subpíxel (sección 4.5)
	link, filter_stubs	vinculación simple de trayectorias en la pestaña <i>Procesar</i> (no se usa en el seguimiento con N fijo)
SciPy (1.18)	optimize.linear_sum_assignment	asignación óptima candidato-robot (algoritmo húngaro)
	spatial.cKDTree	búsqueda de vecinos para confirmar robots nuevos por persistencia
	signal.savgol_filter	derivadas suavizadas: velocidad y ω
	sparse.coo_matrix,	mínimos cuadrados del ángulo acumulado
	sparse.linalg.spsolve, sparse.linalg.cg	(gradiente conjugado con enlaces largos)
NumPy (2.5)	fft.fft, fft.ifft	armónicos de la firma de rotación y correlación circular
	arreglos,	interp,
	savez_compressed, load	todo el cálculo numérico, interpolación de huecos y sesiones .npz
	linalg.lstsq, sliding_window_view	median,
pandas (3.0)	DataFrame, pivot, to_csv, read_csv	ajuste de círculos (Kâsa), rechazo por MAD y mediana móvil del recinto
		tablas de candidatos y de resultados, formato ancho, CSV
PySide6 (Qt 6.11)	QMainWindow, QTabWidget, widgets	interfaz
	de formulario	
	QThread + señales	tareas largas sin congelar la ventana
	QTimer	reproducción del video
Matplotlib	QImage, QPainter	visor con zoom y edición del rectángulo
	imshow, histogram2d (NumPy)	solo en <code>comparar_ensayos.py</code> : mapas de ocupación

Por qué trackpy solo para localizar. `trackpy.locate` es robusto para encontrar máximos y refinarlos a precisión subpíxel, y por eso se usa. Su vinculador (`link`) no sabe que la cantidad de robots es fija: ante un robot que desaparece unos cuadros crea una trayectoria nueva, y ante

un falso positivo, también. Por eso el seguimiento del ensayo usa un algoritmo propio con N fijo (sección 4.6), y `link` queda solo como opción rápida en *Procesar*.

Por qué OpenCV para el video. `cv2.VideoCapture` usa FFmpeg internamente: lee cualquier formato habitual y permite acceso aleatorio por número de cuadro, que necesitan el reproductor, la calibración (cuadros saltados) y la revisión de cuadros con problemas.

3 Escala de tiempo: videos acelerados

Dos frecuencias distintas. Hay que distinguir

- f_a : frecuencia de *reproducción* del archivo (29,97 cuadros/s). Solo define a qué velocidad se ve el video; no tiene significado físico.
- f_r : frecuencia de *captura* en tiempo real, es decir, cuántos cuadros corresponden a 1 s del ensayo. Es la única que entra en los cálculos.

Con el cuadro n contado desde el inicio del archivo, el tiempo real y el factor de aceleración son

$$t_n = \frac{n}{f_r}, \quad \Delta t = \frac{1}{f_r}, \quad A = \frac{f_a}{f_r}. \quad (1)$$

En este informe $f_r = 3$ cuadros/s, valor definido en `config.tex` (`\CuadrosPorSegundo`): entre dos cuadros consecutivos pasan $\Delta t = 0,33$ s reales y el video se reproduce $A = 9,99$ veces más rápido que el ensayo. Todos los tiempos de este documento (gráficos, tablas y equivalencias entre cuadros y segundos) se calculan con ese valor.

Qué depende de f_r y qué no. El análisis completo trabaja en cuadros: detecta, sigue y mide ángulos cuadro por cuadro, y en ningún momento usa el tiempo. Recién al final se pasa a unidades físicas. Si s es la escala espacial [mm/px] y d/dn la derivada respecto del número de cuadro,

$$\vec{v} = s f_r \frac{d\vec{r}_{px}}{dn}, \quad \omega = f_r \frac{d\theta}{dn}. \quad (2)$$

Por lo tanto:

- posiciones, trayectorias, desplazamientos, ángulos θ y estados de seguimiento **no** dependen de f_r ;
- tiempos, velocidades y velocidades angulares son proporcionales a $1/f_r$ o a f_r : un error de f_r se traslada en la misma proporción a $|\vec{v}|$ y a ω ;
- cambiar f_r **no** obliga a repetir el análisis: en la aplicación, la cinemática se recalcula en segundos; en el informe, `generar_informe.py` reescala el CSV (sección 5).

La incertidumbre relativa de las velocidades combina la de la escala espacial y la de la temporal:

$$\frac{\delta v}{v} = \sqrt{\left(\frac{\delta s}{s}\right)^2 + \left(\frac{\delta f_r}{f_r}\right)^2}, \quad \frac{\delta \omega}{\omega} = \frac{\delta f_r}{f_r}. \quad (3)$$

Cómo se determina f_r . Los archivos de la cámara no guardan la marca de tiempo de cada cuadro (la pista de subtítulos DJI solo trae datos de exposición, uno por segundo), así que f_r no se puede leer del video. Hay tres formas de obtenerlo, de mejor a peor:

1. **Configuración de la cámara.** En un *time-lapse* $\times 10$, cada segundo de video (29,97 cuadros) cubre 10 s reales, es decir $f_r = f_a/10 = 2,997$ cuadros/s.

2. **Duración real medida.** $f_r = N_{\text{cuadros}}/T_{\text{real}}$, con T_{real} cronometrado (o tomado del registro del sensor de presión, que tiene su propio reloj y se graba en simultáneo).
3. **Duración nominal del nombre** (`_60`). No conviene: es la duración prevista, no la medida, y en los ensayos del proyecto difiere hasta 12 % de la real (ver el informe de resultados de los ensayos).

Consecuencias del *time-lapse*. Muestrear a f_r en lugar de a 30 Hz cambia lo que se puede medir:

- **Resolución temporal.** Un fenómeno más rápido que la frecuencia de Nyquist, $f_r/2 = 1,5$ Hz, no se puede resolver: un choque que dura menos de $\Delta t = 0,33$ s aparece, a lo sumo, como un salto entre dos cuadros.
- **Desplazamiento por cuadro.** Entre cuadros consecutivos los robots se mueven f_a/f_r veces más que en un video en tiempo real. El seguimiento acepta, como máximo, un apartamiento de $0,625 r_{\text{ext}}$ por cuadro respecto de la posición predicha, lo que equivale a un cambio de velocidad de $0,625 (D/2) f_r \approx 31$ mm/s entre dos cuadros; en los ensayos medidos la rapidez típica es de pocos mm/s, muy por debajo de ese límite.
- **Rotación por cuadro.** El giro entre dos cuadros que se comparan debe ser menor que media vuelta para no confundir su sentido: $|\omega| < 180^\circ \cdot f_r = 540^\circ/\text{s}$.
- **Parámetros en cuadros.** Las ventanas y tolerancias del método están definidas en cuadros, no en segundos, porque el detector y el seguidor trabajan cuadro a cuadro; su duración real depende de f_r (tabla 4).

Tabla 4: Parámetros del método expresados en cuadros y su equivalente en tiempo real con $f_r = 3$ cuadros/s.

Parámetro	Cuadros	Tiempo real
Paso de muestreo Δt	1	0,33 s
Ventana de Savitzky–Golay (velocidad y ω)	7	2,3 s
Hueco máximo que se interpola sin advertencia de error	15	5,0 s
Persistencia exigida a un robot nuevo (75 % de)	20	6,7 s
Comparaciones de rotación (desfasajes)	1, 2, 4, 8, 16	hasta 5,3 s
Flecha de velocidad dibujada (desplazamiento en)	7,5	2,5 s

En la aplicación. En la pestaña *Original*, el grupo *Escala de tiempo* tiene la casilla *Video acelerado (time-lapse)* y el campo $\langle f_r \rangle \text{ cuadros} = 1 \text{ s real}$ (por defecto, 3). El botón *Calcular desde la duración real...* pide la duración medida en minutos y calcula $f_r = N_{\text{cuadros}}/T$; el nombre del archivo se muestra solo como referencia, por ser nominal. Debajo se indica cuánto dura un cuadro en tiempo real, el factor A y la duración real del archivo. El reproductor sigue mostrando el video a f_a , pero la etiqueta de tiempo (*cuadro i / $N \cdot t$*) está en tiempo real. Si se cambia f_r con un seguimiento ya hecho, solo se recalcula la cinemática. Al abrir una sesión guardada (`.npz`) con otra escala, se aplica la escala vigente y se avisa. Desmarcar la casilla vuelve a $f_r = f_a$ (videos en tiempo real).

4 Metodología de medición

Para cada robot de la arena y en cada cuadro del video se mide la posición del centro, el vector velocidad, la orientación acumulada θ y la velocidad angular ω , con la cantidad de robots N fija. Las posiciones se expresan respecto del centro del recinto y se convierten a unidades SI con su diámetro

interior, $D_{\text{int}} = 18,5 \text{ cm}$ (secciones 4.2 and 4.3); el diámetro real de los robots, $D = 3,3 \text{ cm}$, fija los parámetros de detección y el radio de contacto con la pared. Los tiempos usan la frecuencia de captura f_r (sección 3). Las figuras de ejemplo de esta sección provienen de un tramo de 180 cuadros de un ensayo con 22 robots (recorte de $700 \times 700 \text{ px}$), expresado con $f_r = 3 \text{ cuadros/s}$ (60 s reales).

4.1 Flujo de procesamiento

El análisis se divide en dos etapas (fig. 1). La etapa A es la única que lee el video; corregir un centro a mano, cambiar parámetros del seguimiento o recortar el tiempo solo vuelve a correr la etapa B, que tarda segundos.

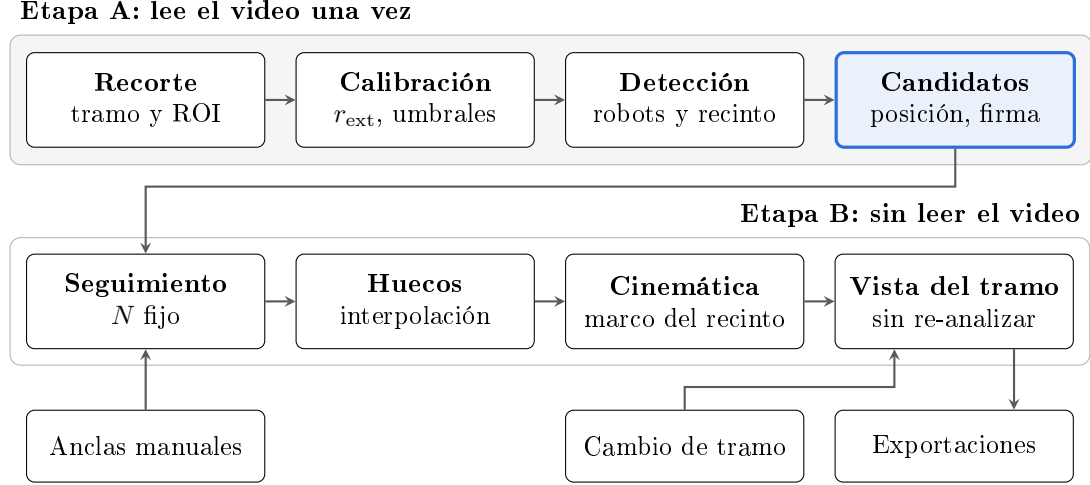


Figura 1: Flujo de procesamiento.

4.2 Sistema de referencia y unidades

Origen en el centro del recinto. Todas las exportaciones (CSV, resumen, informes) usan un sistema fijo al recinto, no a la imagen:

- origen (0,0) en el centro del recinto *en cada cuadro* (el recinto se mueve un poco sobre la placa y la cámara cambia levemente de aumento: sección 4.3);
- x hacia la derecha e y hacia arriba, en la escena real vista desde arriba (con el video espejado ya corregido);
- ángulos (posición polar ϕ , dirección de la velocidad, θ y ω) positivos en sentido antihorario.

Si (X, Y) es el centro de un robot en píxeles del video (Y hacia abajo), $(X_c(n), Y_c(n))$ el centro del recinto en el cuadro n y $s(n)$ la escala de ese cuadro, la posición exportada es

$$x = s(n) \sigma_x [X - X_c(n)], \quad y = -s(n) \sigma_y [Y - Y_c(n)], \quad (4)$$

con $\sigma_x = -1$ si el video está espejado respecto del eje vertical (*horizontal*), $\sigma_y = -1$ si lo está respecto del horizontal (*vertical*) y $+1$ en otro caso. De (x, y) salen la distancia al centro $r = \sqrt{x^2 + y^2}$, el ángulo polar $\phi = \text{atan2}(y, x)$ y la distancia del centro del robot a la pared, $d_{\text{pared}} = R_{\text{int}} - r$, con $R_{\text{int}} = D_{\text{int}}/2$. Un robot apoyado en la pared tiene idealmente $d_{\text{pared}} = D/2$ (sección 4.3).

La superposición sigue en coordenadas de imagen. El análisis (detección, seguimiento, ángulos) se hace sobre la imagen tal como está, para que lo dibujado coincida con el video. Solo al exportar se aplica la ec. (4). Las columnas **x_video_px**, **y_video_px** conservan el centro en píxeles del archivo y **xc_video_px**, **yc_video_px** el centro del recinto de ese cuadro, de modo que la transformación siempre se puede rehacer.

Video espejado. Algunas cámaras graban la escena reflejada. Una reflexión invierte la quiralidad: un giro horario se ve antihorario. Con la opción *Video espejado* de la pestaña *Seguimiento* la ec. (4) refleja la posición respecto del centro del recinto, y θ y ω cambian de signo; las velocidades, al derivarse de (x, y) , quedan corregidas solas. El CSV lleva la columna **espejo**. Los videos de este proyecto están espejados respecto del eje vertical (*horizontal*).

Tabla 5: Magnitudes exportadas (sistema del recinto).

Magnitud	Símbolo	Unidad	Definición
Tiempo real	t	s	$t = n/f_r$, con n el número de cuadro
Posición	x, y	mm	ec. (4): origen en el centro del recinto, y hacia arriba
Posición polar	r, ϕ	mm, °	$r = \sqrt{x^2 + y^2}$, $\phi = \text{atan2}(y, x)$
Distancia a la pared	d_{pared}	mm	$R_{\text{int}} - r$ (centro del robot a la pared)
Velocidad	v_x, v_y	mm/s	$dx/dt, dy/dt$ (Savitzky–Golay)
Rapidez	$ \vec{v} $	mm/s	$\sqrt{v_x^2 + v_y^2}$
Dirección	φ	°	$\text{atan2}(v_y, v_x)$; $0^\circ = +x$
Componente radial	v_r	mm/s	$(xv_x + yv_y)/r$; > 0 = se aleja del centro
Componente tangencial	v_t	mm/s	$(xv_y - yv_x)/r$; > 0 = circula antihorario
Orientación	θ	°	giro acumulado desde el primer cuadro del tramo
Velocidad angular	ω	°/s, rad/s	$d\theta/dt$

4.3 El recinto: detección, marcado manual y escala

El recinto es un anillo de color (verde azulado) de diámetro interior $D_{\text{int}} = 18,5\text{ cm}$ y exterior $D_{\text{ext}} = 19,5\text{ cm}$ (valores editables en la aplicación). Cumple dos funciones: su centro es el origen de coordenadas y su radio en píxeles fija la escala espacial.

1. Máscara del anillo. La imagen se pasa a HSV. El anillo es la estructura saturada dominante: se toman los píxeles con saturación $S \geq \max(60, S_{\text{Otsu}})$ y brillo $V \geq 30$, se busca el tono más frecuente entre ellos (histograma circular de H suavizado en ± 12) y se conservan solo los píxeles con ese tono ± 12 . Los LEDs rojos y las pilas amarillas tienen otro tono y quedan afuera; una apertura morfológica de 3×3 elimina píxeles sueltos (fig. 2a). Si no hay ninguna estructura saturada, se usa un círculo de Hough sobre la imagen en grises y la detección queda marcada como dudosa.

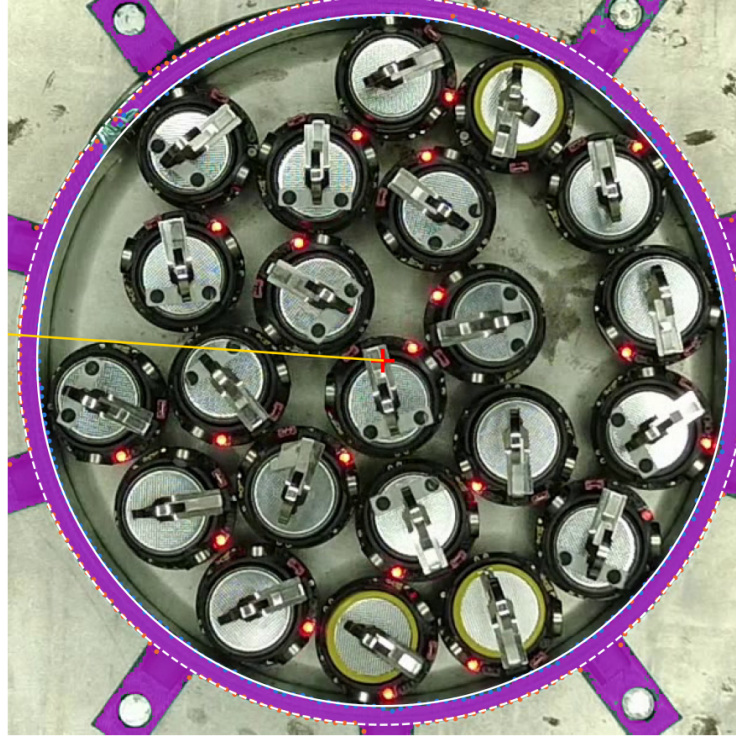
2. Bordes por rayo. Desde un centro provisorio (el de un círculo de Hough sobre la máscara, o el centro del recorte) se trazan 720 rayos (cada $0,5^\circ$) con paso de $0,5\text{ px}$. En cada rayo, el primer tramo de píxeles del anillo da el borde interior y su final el exterior. Se descartan los tramos demasiado largos (brazos de sujeción) y el borde exterior cuando el tramo toca el borde de la imagen. Cada borde se refina a precisión subpíxel en el perfil de saturación: es el punto donde S cruza el valor medio entre el piso y el anillo, interpolado linealmente (fig. 2c).

3. Ajuste de dos círculos concéntricos. Con los puntos interiores (x_i, y_i) y exteriores (x_j, y_j) se resuelve por mínimos cuadrados lineales el sistema (ajuste algebraico de Kåsa)

$$x_i^2 + y_i^2 = 2a x_i + 2b y_i + c_{\text{int}}, \quad x_j^2 + y_j^2 = 2a x_j + 2b y_j + c_{\text{ext}}, \quad (5)$$

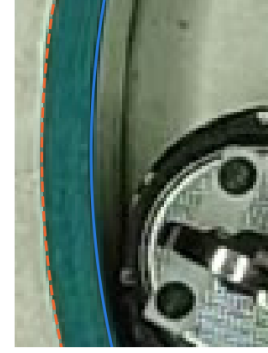
con centro común (a, b) y radios $r_{\text{int}} = \sqrt{c_{\text{int}} + a^2 + b^2}$, $r_{\text{ext}} = \sqrt{c_{\text{ext}} + a^2 + b^2}$. Se itera descartando los puntos con residuo $|\rho - \text{mediana}| > 2,5\text{ MAD}$ (robots que tapan el borde, brazos), y el centro provisorio se reemplaza por el ajustado hasta que se mueve menos de $0,05\text{ px}$. Indicadores de calidad: residuo cuadrático medio del borde interior, cobertura (fracción de rayos aceptados) y el cociente $r_{\text{ext}}/r_{\text{int}}$, que debe valer $D_{\text{ext}}/D_{\text{int}} = 1,0541$ porque no depende de la escala. Se considera confiable con residuo $\leq 3\text{ px}$, cobertura $\geq 50\%$ y cociente dentro de $\pm 0,03$.

(a) máscara del anillo (magenta), bordes por rayo y círculos ajustados



● borde interior (por rayo) ● borde exterior (por rayo)

(b) detalle del borde izquierdo



(c) saturación sobre el rayo amarillo

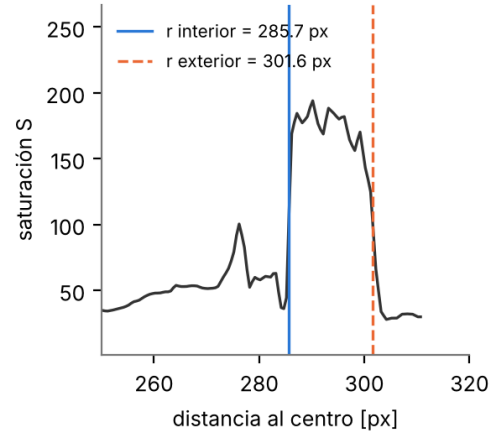


Figura 2: Detección del recinto en un cuadro del ensayo 24/09. (a) Máscara del anillo (magenta), bordes por rayo (puntos) y los dos círculos ajustados (blanco); cruz roja: centro. (b) Detalle: el borde interior (azul) es el del canto superior del anillo; la franja gris clara de adentro es la cara interna de la pared, que se ve por perspectiva. (c) Saturación a lo largo del rayo amarillo con los dos bordes subpíxel.

4. Seguimiento temporal. En los videos del proyecto el recinto no está quieto: se desliza sobre la placa entre 1,5 y 4mm durante la primera media hora, y la cámara cambia su aumento en saltos de $\sim 0,5\%$ (reenfoque). Por eso el recinto se mide cada 15 cuadros (5,0s) durante el mismo recorrido del video que hace el análisis (cuesta $\sim 0,1$ s por medición). Cada medición parte del resultado anterior. Las muestras malas (residuo, cobertura o radio fuera de rango) se descartan, las demás se suavizan con una mediana móvil de 5 muestras (que conserva los saltos de aumento) y se interpolan linealmente a todos los cuadros: así cada posición se refiere al centro y a la escala de su propio cuadro.

5. Marcado manual. Si la detección falla o es dudosa, el círculo se define a mano: con *Marcar a mano* activado, cada clic sobre la imagen agrega un punto del borde interior del anillo, y con 3 o más puntos (mejor 6 a 8, repartidos en todo el contorno) *Ajustar círculo* resuelve el mismo ajuste de Kâsa. Si además hay mediciones automáticas, el círculo manual (marcado en el cuadro n_0) puede seguir el movimiento detectado:

$$\vec{c}(n) = \vec{c}_{\text{manual}} + \vec{c}_{\text{auto}}(n) - \vec{c}_{\text{auto}}(n_0), \quad r(n) = r_{\text{manual}} \frac{r_{\text{auto}}(n)}{r_{\text{auto}}(n_0)}. \quad (6)$$

6. Escala espacial. La aplicación ofrece tres fuentes (*Escala desde*):

- **Recinto** (por defecto): $s(n) = \frac{D_{\text{int}}/2}{r_{\text{int}}(n)}$ [mm/px], recalculada en cada cuadro, de modo que sigue los cambios de aumento de la cámara. Es la referencia más grande de la imagen (~ 570 px de diámetro), así que un error de 1 px en el borde pesa solo 0,2%.

- **Recinto con corrección de perspectiva:** $s(n)$ multiplicada por κ (sección 4.3).
- **Diámetro del robot** (comportamiento anterior): $s = D/(2r_{\text{ext}}^{\text{robot}})$, constante. Subestima el tamaño del robot en la imagen, porque $r_{\text{ext}}^{\text{robot}}$ es el borde del *anillo oscuro* que usa el detector (≈ 47 px) y no el borde del cuerpo (≈ 51 px): sobreestima s en un 8–15 % según el diámetro cargado.

Sin recinto definido, el origen es el centro del recorte analizado y la escala sale del robot; los mensajes y el JSON de cada exportación lo indican.

7. Perspectiva y radio de contacto. La cámara está a una altura finita H : un punto a altura z sobre la placa se ve ampliado en $H/(H - z)$ respecto de la placa. El canto del anillo (donde se mide r_{int}) y la parte superior de los robots (donde se mide su centro) están a alturas distintas, por eso la franja gris de la fig. 2b (la cara interna de la pared vista en escorzo). La consecuencia medible es el *radio de contacto*: el percentil 99,5 de r , es decir, la distancia al centro de los robots apoyados en la pared, debería ser $R_{\text{int}} - D/2$. La aplicación calcula

$$\kappa = \frac{R_{\text{int}} - D/2}{q_{99.5}(r)}, \quad (7)$$

lo informa siempre (JSON y resumen) y solo lo aplica con *Recinto + corrección de perspectiva*. En los ensayos del proyecto $\kappa \approx 0,975$: sin corregir, las posiciones cerca de la pared y las velocidades quedan 2,5 % altas. La corrección supone que los robots que más se alejan del centro tocan la pared con su diámetro completo y depende del D cargado (con $D = 35$ mm en lugar de 33 mm, κ baja 1,3 %); por eso no se aplica por defecto y se reporta como incertidumbre sistemática.

8. Velocidades en el sistema del recinto. v_x y v_y se obtienen derivando con Savitzky–Golay (sección 4.7) las posiciones $x(t)$, $y(t)$ ya expresadas en el sistema del recinto. Son velocidades *respecto del recinto*: si el recinto se desliza, ese desplazamiento no aparece como movimiento de los robots (es del orden de 0,003 mm/s, despreciable frente a la rapidez típica). Los saltos de aumento de la cámara, que en píxeles parecerían un desplazamiento brusco de todos los robots, se cancelan porque cada cuadro usa su propia $s(n)$.

4.4 Calibración automática del tamaño

Visto desde arriba, el robot es un disco claro (la pila) rodeado por un anillo oscuro (la placa). Se usa una plantilla con un disco de radio $r_{\text{int}} = 0,75 r_{\text{ext}}$ y un anillo hasta r_{ext} , y se barre r_{ext} : la respuesta media de los N picos más fuertes es máxima cuando la plantilla coincide con el robot real (fig. 3). A partir de r_{ext} se derivan la escala de análisis ($24/r_{\text{ext}}$, para trabajar siempre con robots de unos 24 px de radio), la separación mínima ($1,5 r_{\text{ext}}$) y el radio de búsqueda del seguimiento ($0,625 r_{\text{ext}}$). Los umbrales de gris se fijan como fracción del contraste medido entre anillo y disco, por lo que no dependen de la exposición.

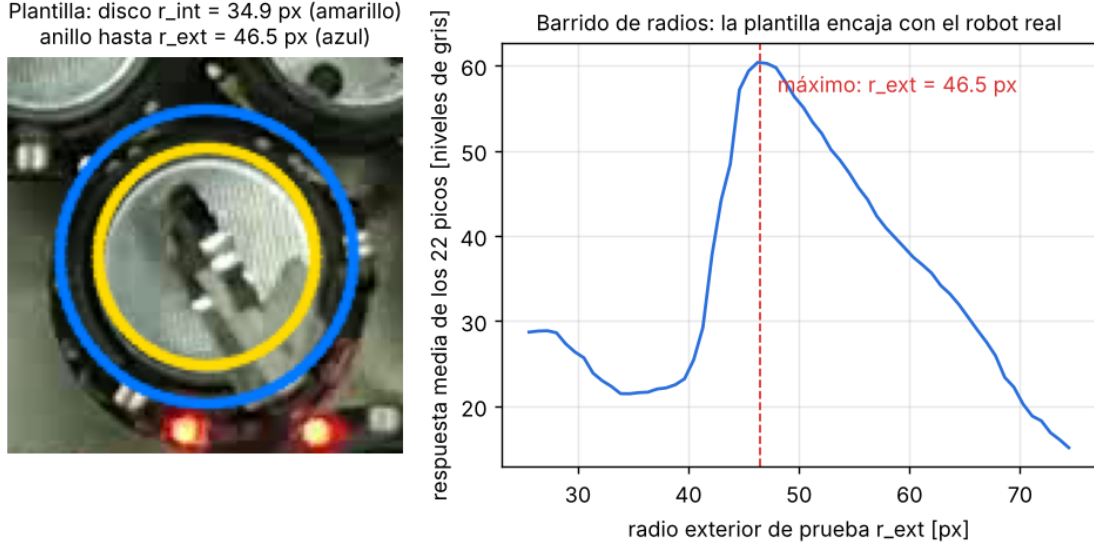


Figura 3: Calibración: plantilla sobre un robot y respuesta en función del radio de prueba.

4.5 Detección

Filtro de anillo. Para cada píxel se calculan el gris medio del disco y del anillo con núcleos circulares normalizados, y se combinan como

$$R(x, y) = (\mu_{\text{disco}} - \mu_{\text{anillo}}) \text{ clip}\left(\frac{G_{\text{max}} - \mu_{\text{anillo}}}{G_{\text{max}}}, 0, 1\right). \quad (8)$$

El primer factor exige contraste; el segundo, que el anillo sea oscuro en valor absoluto. R es alto solo en los centros de los robots. En el código (código 1), las medias locales son dos convoluciones `cv2.filter2D` con núcleos normalizados a suma 1, calculadas sobre la imagen ya reducida a la escala de análisis (los robots quedan con unos 24px de radio, lo que hace el costo independiente de la resolución del video).

```

1 def _disc_ring_kernels(r_in, r_out):
2     R = int(np.ceil(r_out))
3     yy, xx = np.mgrid[-R:R + 1, -R:R + 1]
4     rr = np.hypot(xx, yy)
5     disc = (rr <= r_in).astype(np.float32)
6     ring = ((rr > r_in) & (rr <= r_out)).astype(np.float32)
7     return disc / disc.sum(), ring / ring.sum() # mean filters
8
9 def robot_response(gray, p):
10     """Matched-filter map (uint8) at analysis scale."""
11     ki, kr = _disc_ring_kernels(p.r_in * p.scale, p.r_out * p.scale)
12     g = gray.astype(np.float32)
13     m_disc = cv2.filter2D(g, -1, ki, borderType=cv2.BORDER_REFLECT)
14     m_ring = cv2.filter2D(g, -1, kr, borderType=cv2.BORDER_REFLECT)
15     gate = np.clip((p.ring_gray_max - m_ring) / p.ring_gray_max, 0.0, 1.0)
16     return np.clip((m_disc - m_ring) * gate, 0, 255).astype(np.uint8)

```

Código 1: Filtro de anillo (core/detection.py).

Localización. `trackpy.locate` busca los máximos locales de R y devuelve su centroide con precisión subpíxel. Se aplica sobre R y no sobre la imagen porque la pila no es una mancha uniforme: la palanca metálica y los reflejos corren el centroide. Los argumentos salen de la calibración: el diámetro de la mancha buscada ($1.4r_{\text{int}}$ en la escala de análisis, redondeado a impar, porque el pico de R es más angosto que el disco), la separación mínima entre centros y `minmass`. Las coordenadas devueltas se dividen por la escala para volver a píxeles del video (código 2).

```

1 f = tp.locate(img, p.trackpy_diameter(), minmass=p.minmass,
2             separation=max(1.0, p.separation * p.scale), invert=False)
3 f["x"] = f["x"] / p.scale          # back to video pixels
4 f["y"] = f["y"] / p.scale

```

Código 2: Llamada a trackpy (core/detection.py, función locate).

Validación. Un hueco de piso rodeado de robots también genera un pico, pero su “anillo” está formado por los vecinos y no es cerrado. Se trazan 360 rayos desde el centro y se exige que al menos el 82 % encuentre un píxel oscuro dentro de la banda del anillo (figs. 4 and 5). `cv2.warpPolar` desenrolla la imagen alrededor del candidato: cada fila es un rayo y cada columna un radio, de modo que la cobertura es una sola operación vectorizada (código 3).

```

1 def ring_coverage(gray, x, y, r0, r1, dark):
2     """Fraction of 360 rays whose darkest pixel in the band [r0, r1) is below `dark`."""
3     r1i, r0i = int(np.ceil(r1)), int(max(0, np.floor(r0)))
4     pol = cv2.warpPolar(gray, (r1i + 1, 360), (float(x), float(y)), r1i + 1,
5                          cv2.WARP_POLAR_LINEAR)          # rows = angle, cols = radius
6     return float((pol[:, r0i:r1i].min(axis=1) < dark).mean())

```

Código 3: Cobertura angular del anillo oscuro (core/detection.py).

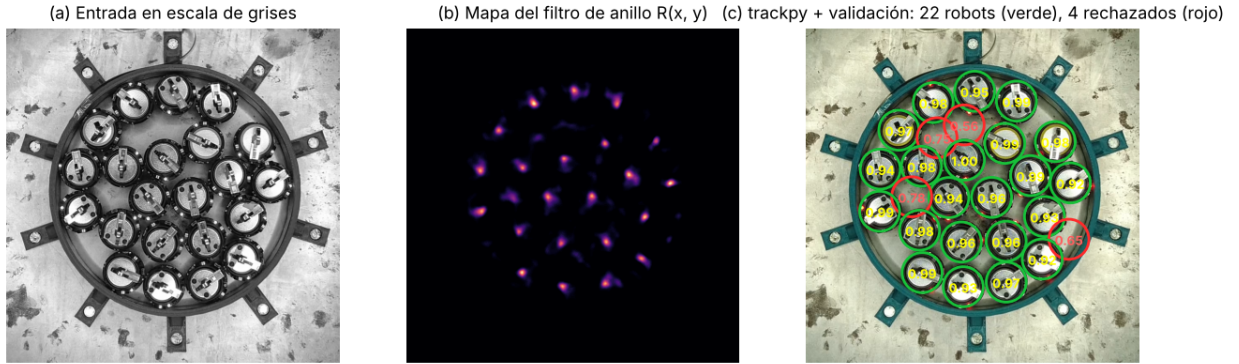


Figura 4: (a) Entrada. (b) Mapa R . (c) Candidatos con su cobertura: aceptados (verde) y huecos rechazados (rojo).

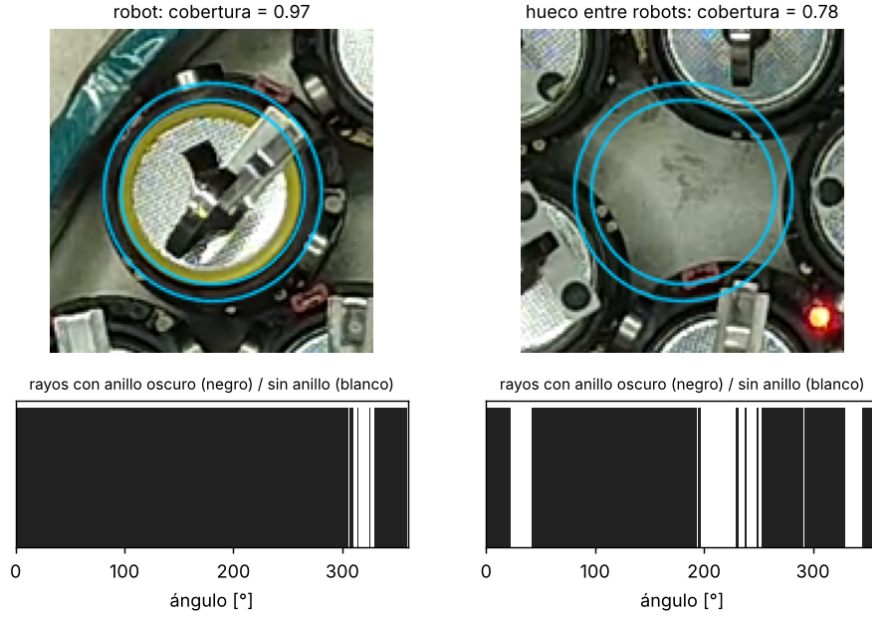


Figura 5: Cobertura del anillo: un robot cierra el anillo en casi todos los rayos; un hueco, no.

4.6 Seguimiento con N fijo

En cada cuadro se predice la posición de cada robot (última posición más velocidad media reciente) y se asignan predicciones y candidatos minimizando la distancia total (algoritmo húngaro), solo dentro del radio de búsqueda. A los robots sin pareja se les ofrecen candidatos con umbrales relajados (cobertura $\geq 0,70$ y luego $\geq 0,55$), pero solo cerca de su predicción. Un robot nuevo solo se acepta si persiste en el 75 % de los 20 cuadros (6,7s) siguientes. Los huecos acotados se interpolan linealmente y cada punto queda con un estado (tabla 6). Un centro marcado a mano (ancla) es una restricción fija desde la que el seguimiento continúa (fig. 6).

El seguimiento corre en dos etapas. La etapa A recorre el video una sola vez y guarda, para cada cuadro, todos los candidatos con un umbral permisivo (cobertura $\geq 0,45$) y su firma de rotación. La etapa B trabaja solo sobre esa tabla: parte del cuadro con mejor detección y avanza hacia adelante y hacia atrás. Cada robot guarda su última posición y una velocidad suavizada exponencialmente ($\alpha = 0,5$); la predicción extrapola esa velocidad, limitada a `max_gap` cuadros para no alejarse demasiado en un hueco. La asignación usa `scipy.optimize.linear_sum_assignment` con costo igual a la distancia, y costo prohibitivo fuera del radio de búsqueda, que crece con la cantidad de cuadros sin detección (código 4).

```

1 def predict(self, frame, max_gap):
2     dt = frame - self.f                                # signed: also runs backwards
3     eff = np.sign(dt) * np.minimum(np.abs(dt), max_gap) # damp long extrapolations
4     return self.xy + self.v * eff[:, None], np.abs(dt)
5
6 def _gated_assign(pred, gates, cand):
7     d = np.linalg.norm(pred[:, None, :] - cand[None, :, :], axis=2)
8     cost = np.where(d <= gates[:, None], d, _BIG)        # outside the gate = forbidden
9     r, c = linear_sum_assignment(cost)                   # Hungarian algorithm
10    return [(i, j) for i, j in zip(r, c) if cost[i, j] < _BIG]
11
12 # per frame: manual anchors first, then the strict and the relaxed coverage levels
13 for cov_thr, gate_factor, status in levels:             # (0.82, 1, DETECTED), (0.70, 0.6, RELAXED), ...
14     ks = np.flatnonzero(st.has & ~done)
15     js = np.flatnonzero(~used & (cov >= cov_thr))
16     for a, b in _gated_assign(pred[ks], base_gate[ks] * gate_factor, xy[js]):
17         ... # store position, status and candidate index; update the velocity

```

Código 4: Predicción y asignación con compuerta (core/tracking.py, resumido).

Tabla 6: Estados de cada posición (columna **status**).

Estado	Significado
detectado	asignado con el umbral estricto
detectado (umbral relajado)	asignado con cobertura 0,55–0,82 cerca de la predicción
manual	centro marcado a mano
interpolado	hueco de hasta 15 cuadros (5,0s) entre dos detecciones (advertencia)
predicho (revisar)	hueco más largo o sin detección de un lado (error)
perdido	sin ninguna posición

Cuadro 105: robot 12 interpolado (naranja), robot 5 con huecos de prueba

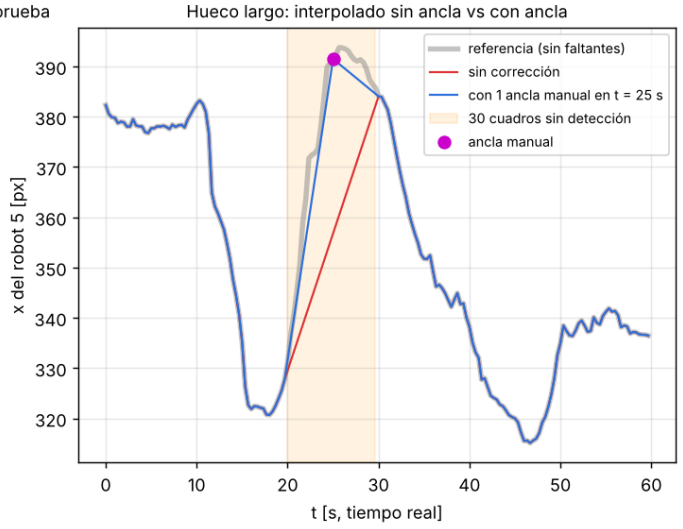
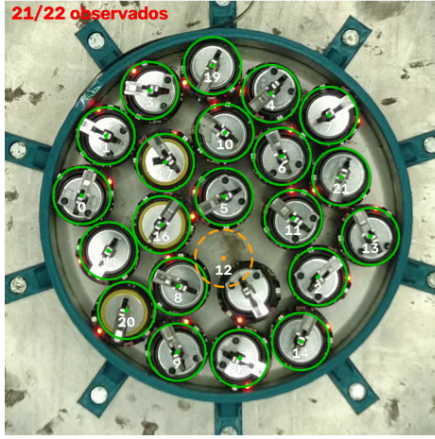


Figura 6: Faltantes simulados: robot interpolado (naranja) y efecto de una sola ancla manual en un hueco de 30 cuadros (10s). Tiempo real con $f_r = 3$ cuadros/s.

4.7 Velocidad

El centro detectado fluctúa entre 1 y 5 px por cuadro; una diferencia finita multiplicaría ese ruido por f_r . Por eso la velocidad se obtiene con un filtro de Savitzky–Golay: para cada instante se ajusta por mínimos cuadrados un polinomio de grado 2 a los cuadros vecinos (ventana de 7 cuadros (2,3s) por defecto) y se toma su derivada analítica,

$$v_x(t_i) = \left. \frac{d\hat{p}_i}{dt} \right|_{t_i}, \quad \hat{p}_i = \arg \min_{p \in \mathbb{P}_2} \sum_{j=-3}^3 [x(t_{i+j}) - p(t_{i+j})]^2, \quad (9)$$

con $t_{i+j} = (i+j)/f_r$, y lo mismo para v_y (fig. 7). Una ventana mayor reduce el ruido pero atenúa los cambios bruscos, como los choques. La ventana w es un número de cuadros (campo *Suavizado* de la pestaña *Seguimiento*): su duración real es w/f_r y crece si la captura es más lenta. En el código, `scipy.signal.savgol_filter` con `deriv=1` y `delta=1/f_r` devuelve directamente la derivada por segundo real; los huecos (NaN) se rellenan por interpolación solo para filtrar y se vuelven a marcar después (código 5). Para la superposición se derivan las posiciones en píxeles de la imagen; para las exportaciones, las posiciones en mm del sistema del recinto (sección 4.3, punto 8), de modo que la escala de cada cuadro queda incluida.

```

1 def _derivative(v, fps, p):
2     """d/dt along axis 0 for each column; NaNs are bridged by interpolation, then restored."""
3     out = np.full_like(v, np.nan, dtype=float)
4     n = len(v)

```

```

5   for k in range(v.shape[1]):                # one column per robot
6       col = v[:, k]
7       ok = np.isfinite(col)
8       if ok.sum() < 2:
9           continue
10      idx = np.arange(n)
11      filled = np.interp(idx, idx[ok], col[ok])
12      w = _odd_window(p.window, n, p.polyorder)
13      d = savgol_filter(filled, w, p.polyorder, deriv=1, delta=1.0 / fps, mode="interp")
14      d[~ok] = np.nan
15      out[:, k] = d
16  return out

```

Código 5: Derivada temporal suavizada (core/kinematics.py).

Robot 2: trayectoria (cian) y v cada 12 cuadros (flecha = desplazamiento en 7,5 cuadros)

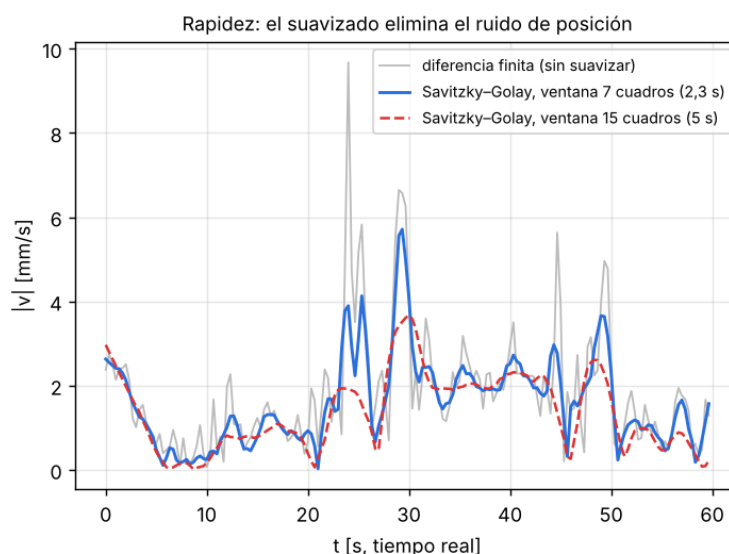
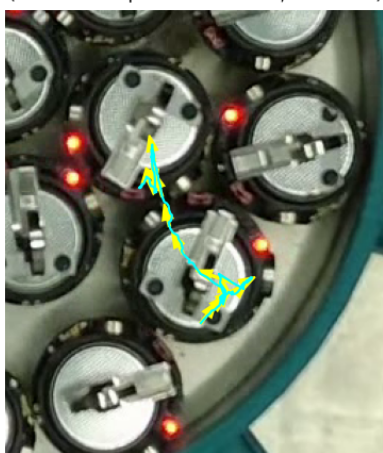


Figura 7: Trayectoria y vector velocidad de un robot (la flecha mide el desplazamiento en 7,5 cuadros, por lo que su largo no depende de f_r); rapidez en tiempo real sin suavizar y con Savitzky-Golay, con $f_r = 3$ cuadros/s.

4.8 Orientación y velocidad angular

Firma de rotación. Alrededor de cada centro la imagen se desenrolla en coordenadas polares; una rotación del robot es entonces un corrimiento a lo largo del eje angular. En dos bandas (pila, $0,15\text{--}0,70 r_{\text{ext}}$, y anillo, $0,75\text{--}0,95 r_{\text{ext}}$) se guarda el perfil angular normalizado como sus primeros $K = 12$ armónicos de Fourier (código 6). Restar la media de cada radio antes de promediar la banda elimina el gradiente radial de iluminación, y normalizar por el desvío hace a la firma insensible al brillo de la escena.

```

1  pol = cv2.warpPolar(gray, (R, 128), (x, y), R, cv2.WARP_POLAR_LINEAR) # 128 angles x R radii
2  for b, (r0, r1) in enumerate(((0.15, 0.70), (0.75, 0.95))):          # battery, PCB ring
3      band = pol[:, int(r0 * R):int(r1 * R)]
4      prof = (band - band.mean(axis=0, keepdims=True)).mean(axis=1)      # angular profile
5      prof = (prof - prof.mean()) / prof.std()
6      sig[b] = np.fft.fft(prof)[1:13] / 128                               # harmonics 1..12

```

Código 6: Firma de rotación de un robot (core/rotation.py, resumido).

Ángulo entre dos cuadros. El giro relativo es el máximo de la correlación circular de las firmas,

$$c(\Delta) = \text{Re} \sum_b \sum_{k=1}^K S_2[b, k] \overline{S_1[b, k]} e^{ik\Delta}, \quad \Delta\theta = \arg \max_{\Delta} c(\Delta), \quad (10)$$

refinado con dos pasos de Newton (error mediano $0,37^\circ$ con rotaciones conocidas). Como $c(\Delta)$ solo tiene armónicos hasta K , una grilla de 128 puntos (por FFT inversa) ubica el máximo y la derivada analítica lo refina (código 7).

```

1 C = (s2 * np.conj(s1)).sum(axis=-2) # cross-spectrum, summed over bands
2 spec[... , 1:K + 1] = C
3 c = np.real(np.fft.ifft(spec, axis=-1)) * 128 # c(Delta) on a 128-point grid
4 d = np.argmax(c, axis=-1) * (2 * np.pi / 128)
5 k = np.arange(1, K + 1)
6 for _ in range(2): # Newton on dc/dDelta = 0
7     e = C * np.exp(1j * k * d[... , None])
8     d1 = np.real((1j * k * e).sum(-1))
9     d2 = np.real(-(k ** 2) * e).sum(-1))
10    d = d - np.where(d2 < 0, d1 / d2, 0.0)
11 angle = -((np.degrees(d) + 180) % 360 - 180) # CCW on screen (warpPolar is clockwise)

```

Código 7: Giro relativo entre dos firmas (`core/rotation.py`, resumido).

Ángulo acumulado. Sumar solo giros entre cuadros consecutivos acumula el error de cada paso. Cada cuadro se compara con los que están 1, 2, 4, 8 y 16 cuadros atrás y todos los ángulos se resuelven juntos por mínimos cuadrados ponderados por la similitud q :

$$\min_{\theta} \sum_{(i,L)} q_{i,L} (\theta_{i+L} - \theta_i - m_{i,L})^2, \quad \theta_0 = 0, \quad (11)$$

El sistema es ralo (`scipy.sparse.coo_matrix`) y se resuelve con las ecuaciones normales. Las comparaciones con similitud $q < 0,8$ o que discrepan más de 30° de la cadena incremental se descartan.

Enlaces de largo alcance. Solo con enlaces cortos el ángulo deriva: cada paso entre cuadros sale levemente sesgado hacia 0 (el gradiente de iluminación no gira con el robot) y el error se suma. En los videos reales, θ discrepaba en $30\text{--}44^\circ$ (mediana) de la comparación directa entre cuadros separados 10 min. Por eso se agregan enlaces de 32, 64, 128... muestras hasta todo el registro (uno cada $L/128$ muestras), nivel por nivel: la rama 2π de cada enlace largo se elige con la solución de los niveles anteriores (tolerancia 90°), con similitud $q \geq 0,8$. Tres pasadas reeligen las ramas con la solución actualizada y una pasada robusta descarta los enlaces con residuo mayor que 15° (la cadena de paso 1 se conserva siempre). Con enlaces largos el sistema deja de ser de banda y se resuelve con gradiente conjugado (`scipy.sparse.linalg.cg`, preconditionador de Jacobi) arrancando de la solución anterior, unas diez veces más rápido que `spsolve`; cuesta ~ 3 s por robot. Validado con bloques de cuadros retenidos: error a 10 min de $0,9^\circ$ (mediana) y 3° (percentil 90), contra $30\text{--}44^\circ$ sin estos enlaces. La herramienta `tools/verificar_giro.py` lo verifica sobre el video (sección 5). Finalmente $\omega = d\theta/dt = f_r d\theta/dn$ con el mismo Savitzky–Golay (fig. 8). La fig. 9 verifica la medición: al girar cada imagen por $-\theta$, los LEDs y los puntos de la placa quedan fijos, por lo que θ describe la orientación del cuerpo del robot (la palanca de la pila gira levemente respecto de él y no sirve como referencia).

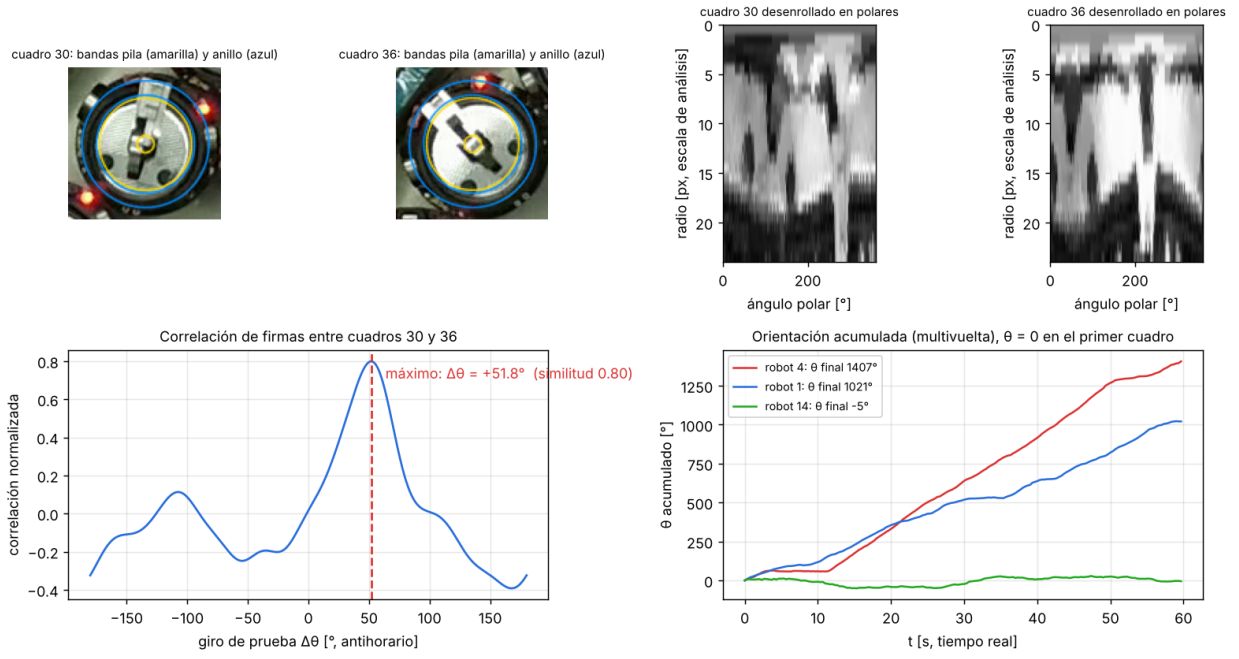


Figura 8: Firma polar en dos cuadros, correlación de las firmas y orientación acumulada de tres robots.

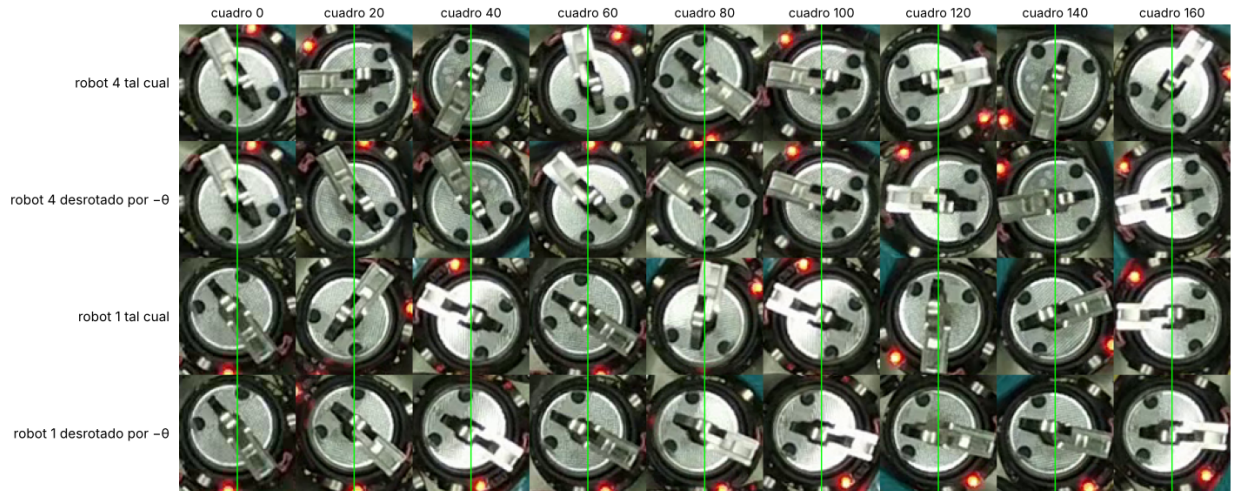


Figura 9: Validación por desrotación: filas pares, las imágenes de la fila anterior giradas por $-\theta$.

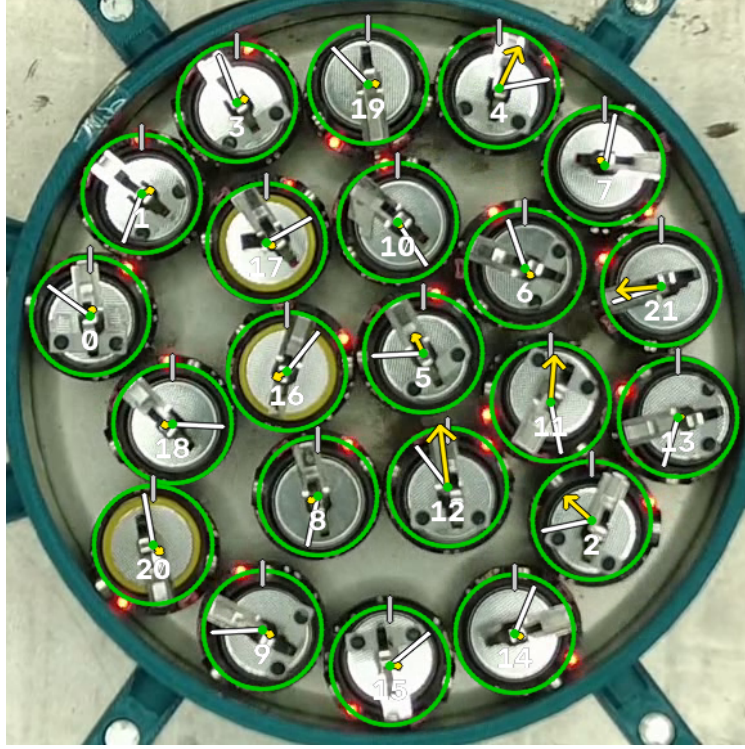


Figura 10: Superposición en la aplicación: estado (color del círculo), velocidad (flecha amarilla, desplazamiento en 7,5 cuadros) y orientación (aguja blanca; la marca a las 12 h es $\theta = 0$).

4.9 Recorte temporal

Si se recorta el tiempo después de analizar, los resultados se toman del análisis completo: las derivadas conservan su precisión en los bordes y θ se re-referencia a cero en el primer cuadro del tramo. Las exportaciones y este informe usan siempre el tramo vigente (sección 5).

4.10 Incertidumbres y limitaciones

- Posición: fluctuación de 1–5 px entre cuadros (0,3–1,6 mm).
- Escala espacial desde el recinto: el radio interior se mide con $\pm 0,2\%$ (residuo de 1–2 px sobre 720 rayos, más la mediana móvil) y el diámetro nominal D_{int} tiene la tolerancia de fabricación del anillo. El cociente $r_{\text{ext}}/r_{\text{int}}$ medido coincide con $D_{\text{ext}}/D_{\text{int}}$ con 0,05 %. Sistemático por perspectiva: 2,5 % (posiciones y velocidades altas si no se aplica κ , sección 4.3).
- Origen: el centro del recinto se mide con unos 0,3 px (0,1 mm) por muestra; entre muestras (5,0 s) se interpola, de modo que un movimiento brusco del recinto se reparte en ese intervalo.
- Escala desde el robot (opción anterior): sesgo de 8 % a 15 % (borde del anillo oscuro en lugar del cuerpo); no recomendada.
- Escala temporal: un error relativo en f_r pasa sin atenuar a $|\vec{v}|$ y a ω , según la ec. (3); con f_r estimado de la configuración de la cámara o del registro del sensor es del orden de 1 %, con la duración nominal del nombre puede superar el 10 %.
- Velocidad: los cambios más cortos que la ventana de suavizado (2,3 s) se atenúan, y nada más rápido que $\Delta t = 0,33$ s se resuelve.
- Orientación: $\pm 0,4^\circ$ por comparación; con los enlaces de largo alcance, el error acumulado a 10 min es de $0,9^\circ$ (mediana) y 3° (percentil 90); en cuadros sin imagen del robot θ se interpola, y un giro mayor que media vuelta entre dos cuadros comparados no se detecta.

- Huecos: la interpolación lineal se aparta si el robot cambia de dirección; tras un hueco, un candidato aceptado con umbral relajado puede ser un falso positivo cercano a una predicción errónea. Para estadística conviene usar solo `status = detectado`.

5 Procedimiento y exportaciones

Todos los comandos se ejecutan desde la **raíz del repositorio**. En Windows (`cmd` o PowerShell) una línea se continúa con `^` (`cmd`) o `'` (PowerShell); en Linux o macOS, con `\`. Las rutas usan `/`, que Python acepta también en Windows.

5.1 Instalación

```
pip install -r software/vidfetch/requirements.txt
pip install -r software/fsrscope/requirements.txt      (solo para FSRScope)
```

Requiere Python 3.10 o superior. Para compilar los informes hace falta una distribución de L^AT_EX (MiKTeX o TeX Live) con `pgfplots`, `siunitx` y `listings`.

5.2 Analizar un video con la interfaz

```
python software/vidfetch/main.py
```

1. **Abrir** el VC_<código>.MP4 (menú *Archivo*, pestaña *1 · Original*).
2. **Escala de tiempo**. En la misma pestaña, dejar marcada *Video acelerado (time-lapse)* y fijar f_r (sección 3); para los videos del proyecto, 3.
3. **Recorte** (opcional, pestaña *2 · Recortar*): tramo temporal y rectángulo. Los VC ya están recortados a la arena. El rectángulo debe contener el anillo completo o casi completo: el recinto se detecta dentro de él.
4. **Parámetros** (pestaña *4 · Seguimiento*): cantidad de robots N , diámetro real del robot (33 mm), *Video espejado: horizontal*, diámetros del recinto (185 y 195 mm), *Escala desde: Recinto* y la casilla *Detectar el recinto al analizar* marcada.
5. **Analizar video**. En una sola lectura del video se calibra el tamaño de los robots, se detectan los candidatos, se mide el recinto cada 15 cuadros y se hace el seguimiento.
6. **Revisar el recinto**: con *Mostrar recinto y ejes* se dibujan el borde interior (amarillo), el exterior (celeste, punteado), el centro (cruz roja) y los ejes x , y tal como caen sobre la imagen (con el video espejado, $+x$ apunta a la izquierda de la imagen). El cuadro *Recinto* muestra centro, radio, escala, $r_{\text{ext}}/r_{\text{int}}$ contra el esperado, residuo, movimiento del recinto y variación de escala. Si el círculo amarillo no apoya sobre el borde interior del anillo, marcarlo a mano (sección 5.3).
7. **Revisar los robots**: lista *Cuadros con problemas* (botones *Anterior* y *Siguiente*); para corregir un centro, clic sobre el robot y clic en su centro real (ancla). El seguimiento se rehace en segundos, sin volver a leer el video.
8. **Guardar** el análisis (*Guardar análisis...*, `.npz`: incluye el recinto) y exportar (sección 5.4).

5.3 Definir el recinto a mano

1. Ir a un cuadro donde el borde interior del anillo se vea bien (sin robots encima).
2. Pulsar *Marcar a mano (clics en el borde interior)*. Mientras está activo, los clics no seleccionan robots: cada clic agrega un punto (magenta).
3. Hacer al menos 3 clics, mejor 6 a 8 repartidos en todo el contorno, sobre el borde interior del canto del anillo (el límite entre el anillo de color y la franja gris de adentro). *Borrar puntos* empieza de nuevo.
4. *Ajustar círculo a los puntos*: se ajusta el círculo, se recalcula la cinemática y el modo manual se desactiva.
5. Si existe una detección automática, *El círculo manual sigue el movimiento detectado* lo traslada y escala con ella (sección 4.3, punto 5). Sin detección automática, el círculo queda fijo en todo el video.
6. *Descartar el círculo manual* vuelve al automático; *Detectar recinto automáticamente* repite la medición sobre todo el tramo analizado.

Al cargar una sesión guardada antes de que existiera el recinto, la aplicación lo detecta sola (lee el video una vez, 30–90 s).

5.4 Exportaciones

Todas usan el tramo vigente y el sistema del recinto (sección 4.2). Junto a cada CSV se escribe `<nombre>_info.json` con el recinto (centro, radio, escala, cociente de radios, residuo, movimiento, variación de escala, cantidad de muestras), la fuente de la escala, κ medido y aplicado, el radio de contacto, el diámetro del robot, el espejado, f_r , el video y el recorte.

- **Exportar CSV (todos los robots)**: un único archivo; en el diálogo se elige *una fila por robot y cuadro* (formato largo, tabla 7, el que usan los informes) o *una fila por cuadro* (formato ancho: columnas `r00_x_mm`, `r01_x_mm`, ...).
- **Exportar un robot (todos los datos)**: elegir el robot en la lista *Robot* y exportar: todas las columnas de la tabla 7, una fila por cuadro, solo para ese robot.
- **Exportar cada robot en su propio archivo**: elegir una carpeta; se escribe `<video>_robot00.csv` ... `<video>_robot21.csv` y un JSON.
- **Exportar resumen por robot**: una fila por robot (tabla 8).
- **Exportar video con seguimiento**: el tramo con la superposición (robots, velocidad, orientación y, si está marcado, el recinto con sus ejes).
- **Guardar análisis**: sesión `.npz` (candidatos, firmas de rotación, anclas, recinto y parámetros; NumPy comprimido, sin `pickle`).

Tabla 7: Columnas del CSV de seguimiento (formato largo y por robot). Posiciones y velocidades en el sistema del recinto (ec. (4)).

Columna	Unidad	Contenido
frame	—	número de cuadro n en el archivo de video
t_s	s	tiempo real n/f_r
particle	—	identificador del robot ($0 \dots N - 1$), fijo durante el ensayo
status	—	estado de la posición (tabla 6); <code>status_code</code> , su código
x_mm, y_mm	mm	centro del robot; origen en el centro del recinto, y hacia arriba
r_mm, phi_deg	mm, °	coordenadas polares; ϕ antihorario desde $+x$
wall_dist_mm	mm	$R_{\text{int}} - r$: del centro del robot a la pared
vx_mm_s, vy_mm_s	mm/s	velocidad respecto del recinto
speed_mm_s	mm/s	rapidez $ \vec{v} $
vel_dir_deg	°	dirección de $\vec{v}$, antihoraria, $0^\circ = +x$
v_rad_mm_s, v_tan_mm_s	mm/s	componentes radial (> 0 hacia afuera) y tangencial (> 0 antihoraria)
theta_deg	°	orientación acumulada θ , cero en el primer cuadro del tramo
omega_deg_s, omega_rad_s	°/s, rad/s	velocidad angular ω (antihoraria positiva)
rot_status	—	<i>medida, interpolada o no disponible</i>
x_px, y_px, r_px	px	lo mismo que <code>x_mm</code> , <code>y_mm</code> , <code>r_mm</code> sin escalar
vx_px_s, vy_px_s	px/s	velocidad en píxeles (ejes del recinto)
speed_px_s		
mm_per_px	mm/px	escala $s(n)$ del cuadro
xc_video_px, yc_video_px	px	centro del recinto en ese cuadro, en píxeles del video
x_video_px, y_video_px	px	centro del robot en píxeles del video (sin espejar, y hacia abajo)
mass, ring_cov	—	intensidad de la detección y cobertura del anillo
espejo	—	solo si se corrigió el espejado: <code>horizontal</code> o <code>vertical</code>

Tabla 8: Columnas del resumen por robot (en mm si hay escala; en px si no).

Columna	Contenido
frames, duration_s	cuadros y duración real del tramo
pct_position_observed, pct_rotation_measured	% de posiciones observadas y de θ medido (no interpolado)
mean/median/max_speed_mm_s	rapidez media, mediana y máxima
path_length_mm, net_displacement_mm	recorrido total y desplazamiento neto
mean_r_mm	distancia media al centro del recinto
total_rotation_deg, turns	giro neto en grados y en vueltas (antihorario positivo)
mean_omega_deg_s, mean_abs_omega_deg_s	$\bar{\omega}$ con signo y $ \bar{\omega} $
pct_omega_cw	% del tiempo girando en sentido horario ($\omega < 0$)
mean_v_tan_mm_s	$\bar{v}_t$: circulación alrededor del centro (> 0 antihoraria)
pct_wall_layer	% del tiempo en la capa de la pared: $r \geq q_{99.5}(r) - D/2$ (a menos de medio diámetro del radio de contacto)

Leer los CSV en otros programas. El separador es la coma y el decimal el punto. En un Excel configurado en español conviene importarlos con *Datos* $\rightarrow$ *Desde texto/CSV* eligiendo punto como separador decimal; abrirlos con doble clic puede leer 0.52 como texto o fecha. En Python:

```

1 import pandas as pd
2 df = pd.read_csv("datos/video/trayectorias/VP_20262409_1600_22QR_60_robots.csv")
3 r7 = df[(df.particle == 7) & (df.status == "detectado")] # robot 7, observed points only
4 print(r7[["t_s", "x_mm", "y_mm", "speed_mm_s", "omega_deg_s"]].describe())
5 wall = df.r_mm >= df.r_mm.quantile(0.995) - 33 / 2 # wall layer (D = 33 mm)

```

Código 8: Lectura de un CSV exportado (un robot y solo puntos detectados).

5.5 Reexportar sin la interfaz: reexportar.py

Carga una sesión .npz, rehace el seguimiento y la cinemática con las opciones indicadas y escribe los mismos archivos que la interfaz. Es la forma de regenerar las trayectorias de un ensayo:

```

python software/vidfetch/tools/reexportar.py ^
    datos/video/sesiones/VA_20262409_1600_22QR_60_analisis.npz ^
    --video datos/video/recortados/VC_20262409_1600_22QR_60.MP4 ^
    --espejo horizontal --ventana 5 --diametro-robot 33 --escala recinto ^
    --cuadros-por-segundo 3 ^
    --salida datos/video/trayectorias/VP_20262409_1600_22QR_60_robots.csv ^
    --resumen --guardar-sesion

```

Tabla 9: Opciones de reexportar.py.

Opción	Efecto
sesion (obligatoria)	análisis guardado por la app (.npz)
--video	video analizado, si no está en la ruta guardada en la sesión
--salida	CSV largo (por defecto <sesión>_robots.csv); también escribe _info.json
--resumen	escribe además <salida>_resumen.csv
--por-robot CARPETA	un CSV por robot en esa carpeta
--espejo, --ventana,	como en la pestaña <i>Seguimiento</i>
--diametro-robot	
--escala	fuelle de la escala (sección 4.3, punto 6)
recinto perspectiva robot	
--cuadros-por-segundo	f_r
--d-int, --d-ext	diámetros del recinto [mm] (por defecto 185 y 195)
--redetectar, --paso	vuelve a medir el recinto en el video cada paso cuadros (por defecto 15); sin esta opción se usa el guardado en la sesión, y solo se mide si la sesión no lo tiene
--guardar-sesion [NPZ]	guarda la sesión con el recinto (sobre la misma o en NPZ)

5.6 Verificar el giro: verificar_giro.py

Recorta algunos robots del video cada pocos minutos y los muestra tal como se grabaron y girados por $-\theta$ (método anterior y actual). Si θ es correcto, las marcas del robot quedan quietas:

```

python software/vidfetch/tools/verificar_giro.py ^
    datos/video/sesiones/VA_20262409_1600_22QR_60_analisis.npz ^
    --video datos/video/recortados/VC_20262409_1600_22QR_60.MP4 ^
    --robots 3,7,12 --inicio 0 --cada 2.5 --duracion 15

```

Escribe datos/video/verificacion/giro_<código>.png (o --salida).

5.7 Generar el informe de ensayos

Dos scripts de `software/analisis_video/` arman informes/ensayos/informe_ensayos.tex a partir de los CSV (y sus `_info.json`). Por defecto escriben en `informes/ensayos/generado/` y leen el registro de presión de `datos/presion/crudos/`.

Una sección por ensayo. `generar_informe.py` crea `generado/<nombre>/` con la tabla de datos (incluido el recinto), la captura anotada (con el recinto y los ejes), los gráficos y un párrafo automático, y actualiza `generado/lista_ensayos.tex`:

```
python software/analisis_video/generar_informe.py ^
    datos/video/trayectorias/VP_20262409_1600_22QR_60_robots.csv ^
    --video datos/video/recortados/VC_20262409_1600_22QR_60.MP4 ^
    --nombre 20262409_1600_22QR_60
```

Opciones:

- `--cuadros-por-segundo F`: reexpresa el CSV con $f_r = F$ (recalcula $t = n/F$ y multiplica velocidades y velocidades angulares por F/f_r^{CSV} , con f_r^{CSV} deducido de `frame` y `t_s`).
- `--imagen archivo.png --frame n`: usa una imagen ya extraída del cuadro n para la captura, en lugar de leer el video.
- `--titulo`; `--diametro-mm` (por defecto, el del JSON o 33); `--diametro-px` (solo para CSV viejos sin escala).

Si la duración real supera 10 min, los ejes de tiempo van en minutos y las trayectorias se grafican solo para los primeros 5 min. Las observaciones propias van en `generado/<nombre>/notas.tex`, que el script nunca sobrescribe. Los CSV anteriores al recinto (con `x_px` relativo a la esquina del recorte) siguen funcionando.

Comparación entre ensayos. `comparar_ensayos.py` recibe todos los CSV y genera `generado/comparacion/`: tabla comparativa (incluidos recinto, radio de contacto, ϕ , movimiento del recinto y zoom), series temporales con los atascos, distribuciones, rotación por robot, desplazamiento cuadrático medio, mapas de ocupación con la pared y el radio de contacto, densidad radial y, si encuentra los registros, el contraste con el sensor de presión:

```
python software/analisis_video/comparar_ensayos.py ^
    "datos/video/trayectorias/VP_*_robots.csv"
```

El umbral de atasco (percentil 90 de la rapidez suavizado en 30s menor que 0,52 mm/s durante al menos 20s) está al comienzo del script (`FREEZE_P90_MM_S`).

Compilar. Desde `informes/ensayos/` y `informes/aplicacion/`:

```
latexmk -pdf informe_ensayos.tex          (o dos pasadas de pdflatex)
latexmk -pdf informe_aplicacion.tex
```

5.8 Rehacer todo desde cero

Con los videos copiados en `datos/video/recortados/`, para cada `<código>`:

1. Con la interfaz: abrir `VC_<código>.MP4`, fijar parámetros (paso 4 de la lista anterior), *Analizar video*, revisar recinto y robots, *Guardar análisis* como `datos/video/sesiones/VA_<código>_analisis.npz` y exportar el CSV largo como `datos/video/trayectorias/VP_<código>_robots.csv`.

2. O, si la sesión ya existe, sin interfaz: el comando de sección 5.5 (agregar `--redetectar` para volver a medir el recinto).
3. `generar_informe.py` para cada ensayo y `comparar_ensayos.py` con todos.
4. Compilar los dos informes.

Las trayectorias versionadas en el repositorio se generaron con `reexportar.py` y los parámetros del ejemplo anterior, con el recinto medido cada 30 cuadros sobre cuadros extraídos del video (JPEG de alta calidad) y guardado en cada sesión. Sin `--redetectar` se reproducen exactamente; con `--redetectar` el recinto se vuelve a medir cada 15 cuadros sobre el video original y cabe esperar diferencias del orden del ruido de una muestra (0,3px en el centro).

Sesión de análisis (.npz). Guarda candidatos, firmas de rotación, anclas manuales, recinto (muestras automáticas y círculo manual) y parámetros. Al cargarla se aplica la escala de tiempo vigente en la aplicación; las sesiones sin recinto lo miden al cargarse.